Tutorial 5: Matrix operations

Matteo Fasiolo

Tutorial outline

Open `matrix_ops.c`. In this tutorial, you will:

1. represent matrices using a structure and dynamic allocation;
2. access and print matrices stored in row-major order;
3. multiply two matrices;
4. release the dynamically allocated arrays.

We will use small integer matrices so that you can check every result by hand.

A dynamic matrix structure

The supplied file defines:

```
struct matrix {
    int nrow;
    int ncol;
    int *elements;
};
```

The structure stores the matrix dimensions and a pointer to its elements.

For example, a matrix `M` needs space for `M.nrow * M.ncol` integers:

```
M.elements = malloc(M.nrow * M.ncol * sizeof(int));
```

Allocate and initialise two matrices

The program uses

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad B = \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix}.$$

1. Complete **TODO 1**: allocate the element arrays for `A` and `B`.
2. Complete **TODO 2**: copy the supplied values into those arrays.

Both arrays store their elements one row after another.

Row-major indexing

For a matrix with `ncol` columns, entry (i, j) is stored at position

```
i * ncol + j
```

For example, `A` is stored as:

```
1 2 3 4 5 6
```

Therefore, `A[1, 2]` is stored at position

```
1 * 3 + 2 = 5
```

and its value is 6. All indices are zero-based.

Access and print the matrices

1. Complete **TODO 3** in:

```
int get_elem(struct matrix M, int i, int j);
```

It should return entry (i, j) of `M`.

2. Complete **TODO 4** in `print_matrix` using two nested loops and `get_elem`.

3. Compile and run the program. Check that it prints:

```
A:
1 2 3
4 5 6

B:
7 8
9 10
11 12
```

Matrix multiplication

The number of columns of A equals the number of rows of B:

$$(2 \times 3)(3 \times 2) = 2 \times 2.$$

Each entry of $C = AB$ is a dot product. For example,

$$C_{0,0} = 1(7) + 2(9) + 3(11) = 58.$$

In the code:

- `i` selects a row of `A`;
- `j` selects a column of `B`;
- `k` moves along that row and column.

Implement matrix multiplication

The outer two loops are supplied in:

```
void multiply(struct matrix A,
              struct matrix B,
              struct matrix C);
```

Complete **TODO 5**: add a loop over `k` that accumulates

```
get_elem(A, i, k) * get_elem(B, k, j)
```

into `total`.

The final line of the inner loop stores `total` in entry (i, j) of `C`.

Allocate and use the result

The result has:

```
C.nrow = A.nrow;
C.ncol = B.ncol;
```

1. Complete **TODO 6**: allocate `C.elements`.
2. Call `multiply` and print `C`.

Although `C` is copied into the function's local parameter, its copied pointer still points to the elements allocated in `main`. The function therefore fills that same array.

Check the result and release the arrays

The complete output for the product should be:

```
A x B:
58 64
139 154
```

Complete **TODO 7** by releasing the three dynamically allocated arrays.

Optional Challenge: Matrix Powers

For a square matrix Q, define

$$Q^0 = I, \qquad Q^p = Q^{p-1}Q \quad \text{for } p > 0,$$

where I is the identity matrix.

Write a function that creates a new dynamically allocated matrix containing $A^{\texttt{power}}$:

```
int matrix_power(struct matrix A, int power,
                 struct matrix *result);
```

It should return `1` on success and `0` if `A` is not square, `power` is negative, or an allocation fails.

Matrix Powers: Start with the Identity

The function must change all three members of `result`, so it receives the address of that structure. Access its members using `result->nrow`, `result->ncol` and `result->elements`.

First allocate a matrix called `current` with the same dimensions as `A`, then initialise it as the identity matrix:

```
1 0 ... 0
0 1 ... 0
...
0 0 ... 1
```

Thus, if `power == 0`, `current` is already the correct answer.

Matrix Powers: Repeated Multiplication

Repeat `power` times:

1. allocate a new matrix called `next`;
2. compute `next = current * A` using `multiply`;
3. release `current.elements`;
4. assign `next` to `current` by writing `current = next`.

The assignment copies the structure members, including the pointer.

Matrix Powers: Test

Test your function using

$$Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}.$$

The expected result is

```
Q to the power 5:
8 5
5 3
```

Use a newly declared structure for `result`. After a successful call, `result->elements` points to an array allocated by `matrix_power`. When `main` has finished using the result, it must call `free(result.elements)` exactly once.

What is the time complexities for an $n \times n$ matrix and exponent p?